



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 4

Clasificación Multiclase: el Mapa de Facies del Campo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos

✓ Ya tenemos:

- ✓ Regresión lineal: predecir un **número** (Clase 1)
- ✓ Logística: predecir **una de dos** etiquetas (Clase 2)
- ✓ Árboles y **Random Forest**: la no linealidad (Clase 3)
- ✓ El hilo de siempre: métricas + **costo de negocio**

Pero el subsuelo casi nunca pregunta “¿arena sí o no?”.

Pregunta “¿cuál de todas?”.

▶▶ Hoy:

- Clasificar entre **9 facies** a la vez — un dataset nuevo: **Hugoton, Kansas**
- **Codificar** variables: el idioma que los modelos entienden
- Métricas multiclase: matriz 9×9 , macro vs weighted
- ★ El costo de negocio, ahora con **matriz de penalización**

*Y al final, sembramos la pregunta que abre la Clase 5:
¿quién decide las **perillas** del modelo?*

El Problema Multiclase

Cuando dos etiquetas no alcanzan

Ustedes Ya Clasifican en Más de Dos

Piensen en la última vez que describieron ripios en el pozo o un núcleo en el laboratorio.

👁 LO QUE HACEN EN EL CAMPO

El geólogo de pozo no reporta “roca sí / roca no”. Reporta: **arenisca, limolita, lutita, caliza, dolomita...** y discute los casos dudosos con el registro en la mano.

Eso es exactamente **clasificación multiclase**: elegir **una entre varias** categorías, para cada muestra.

- En la Clase 2 lo **simplificamos** a propósito: ¿lutita o no lutita? Dos etiquetas.
- Hoy quitamos la simplificación: el modelo debe elegir entre **9 facies**, pie a pie.
- ✓ La buena noticia: **casi todo lo que saben se recicla** — train/test, accuracy, precision, recall, el bosque.
- ⚠ Lo nuevo: cómo **alimentar** categorías al modelo y cómo **leer** 9 clases a la vez.

¿Qué Cambia al Pasar de 2 a 9 Clases?

Menos de lo que temen, más de lo que parece.

	Binario (Clase 2-3)	Multiclase (hoy)
El modelo responde	lutita / no lutita	una de 9 facies
Probabilidades	una: $P(\text{lutita})$	nueve , y suman 1
Matriz de confusión	2×2 : 4 casillas	9×9 : 81 casillas
Precision / recall	un número de cada	uno por clase (9 pares)
Costo de negocio	FP vs FN	matriz de penalización
El código	<code>.fit()</code> / <code>.predict()</code>	idéntico

- ✓ scikit-learn detecta solo cuántas clases hay en y: el flujo no cambia.
- Lo que sí cambia: **preparar los datos y leer los resultados**. Ahí está la clase de hoy.

El Dataset: Hugoton, Kansas

La primera competencia de ML en geociencias

Un Campo Gigante y una Competencia

Hugoton (Kansas, EE. UU.): uno de los campos de gas más grandes de Norteamérica.

- 🏠 Reservorio en carbonatos y areniscas del **Pérmico** (Grupo Council Grove): ciclos que alternan ambientes **marinos** y **no marinos**.
- 🏆 En 2016 la **SEG** lanzó con estos datos la primera competencia de *machine learning* en geociencias: predecir **facies** desde registros.
- 👥 Cientos de equipos en el mundo compitieron con **estas mismas columnas** que van a usar hoy.

LA TABLA

4 149 filas = intervalos de medio pie

10 pozos con núcleo descrito

5 registros + 2 variables de contexto

1 target: la facies (9 categorías)

*¿Por qué existe el dataset? Describir núcleo es caro: solo algunos pozos lo tienen. El registro eléctrico, en cambio, **lo tienen todos**.*

Acotar el Problema (Como Siempre, Antes de Modelar)

La tabla que llenamos en cada clase, ahora en versión multiclase.

Pregunta de negocio	¿Qué facies hay en cada medio pie de los pozos sin núcleo , usando solo sus registros?
Target	Facies — 9 categorías geológicas
Features	5 registros (GR, ILD_log10, DeltaPHI, PHIND, PE) + contexto (NM_M, RELPOS, Formation)
Métrica de éxito	accuracy y F1 por clase — y al final, la penalización total (costo multiclase)
Récord a batir	el modelo tonto (siempre la facies más común): 23 % de aciertos

Regla de la casa: sin esta tabla no se escribe ni una línea de código.

Primero lo Primero: ¿Qué es una Facies?

Antes de predecir facies, acordemos qué estamos prediciendo.

LA IDEA

Una **facies** es un “tipo de roca”: un paquete de características (grano, textura, minerales, fósiles) que se repiten juntas porque la roca se formó en el **mis-mo ambiente de depósito** — una playa, una laguna, un arrecife.

Analogía: dos lotes de cemento de la misma planta y la misma receta se comportan igual en el pozo. Dos rocas de la misma facies, también.

- ¿Por qué le importa al ingeniero? Porque la facies **controla la porosidad y la permeabilidad**: el mapa de facies es un mapa de **calidad de roca**.
- Con facies se **correlaciona** entre pozos, se decide **dónde completar** y se construye el modelo geológico del yacimiento.
- ✓ La describe un geólogo mirando el **núcleo...** cuando hay núcleo. Nuestro modelo intentará hacerlo **desde los registros**.

Las 9 Facies: Nuestro Menú de Categorías

Tres son de ambiente no marino, seis de ambiente marino. Guarden los colores: nos acompañan toda la clase.

	Código	Qué es	Ambiente
	SS	Arenisca	no marino
	CSiS	Limolita gruesa	no marino
	FSiS	Limolita fina	no marino
	SiSh	Limolita y lutita marina	marino
	MS	Mudstone (caliza de grano muy fino)	marino
	WS	Wackestone (caliza con matriz de lodo)	marino
	D	Dolomita	marino
	PS	Packstone-grainstone (caliza granular)	marino
	BS	Bafflestone (caliza de algas)	marino

→ Ojo geológico: varias son **vecinas de grano**: una limolita gruesa y una fina se parecen *muchísimo*. Recuerden esto cuando veamos los errores del modelo.

Las Variables, Una a Una (Lectura Física)

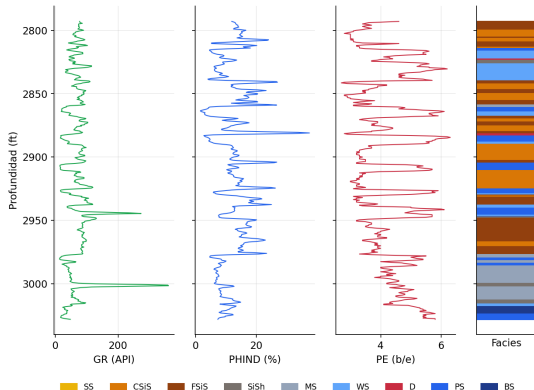
Antes de dárselas al modelo, entendamos qué mide cada una.

Columna	Tipo	Qué nos dice
GR	numérica	Radioactividad natural: las arcillas radian más
ILD_log10	numérica	Resistividad profunda (en log): fluidos y litología
DeltaPHI	numérica	Diferencia entre dos porosidades: sensible a litología
PHIND	numérica	Porosidad promedio neutrón–densidad
PE	numérica	Efecto fotoeléctrico: huella de la mineralogía
NM_M	categorica (2)	Contexto: intervalo no marino (1) o marino (2)
RELPOS	numérica	Posición relativa dentro del ciclo estratigráfico (1 = tope)
Formation	categorica (14)	En qué formación está la muestra (A1 SH, B5 LM, ...)

⚠ Dos columnas **no son números**: NM_M y Formation. Todavía no sabemos metérselas al modelo — ese es el tema de la próxima sección.

Así se Ve un Pozo del Dataset

Pozo SHRIMPLIN: registros y facies



- Cada medio pie tiene sus registros y la facies descrita en el núcleo (la columna de colores).
- Arriba domina lo **no marino** (cafés y amarillos); abajo, los carbonatos **marinos** (azules).
- ✓ Fíjense cómo el GR sube en las facies arcillosas y el PE cambia entre areniscas y calizas: **ahí hay señal**.
- 🎯 La tarea del modelo: **reconstruir la columna de colores** viendo solo las tres curvas.

Primer Vistazo con describe()

Como en cada clase: números resumen primero, interpretación física al lado.

Python

```
url = "https://raw.githubusercontent.com/..."
df = pd.read_csv(url)
print(df.shape)
df[['GR', 'DeltaPHI', 'PHIND',
    'PE']].describe().round(2)
```

>_ Output

```
(4149, 11)
      GR  DeltaPHI  PHIND    PE
count  4149      4149   4149  3232
mean   64.9      4.40   13.2   3.73
std    30.3      5.27    7.1   0.90
min    10.2     -21.83    0.6   0.20
max    361.2     19.31   84.4   8.09
```

- GR máximo **361 API**: lutitas calientes, muy radioactivas. El promedio (65) es roca normal.
- DeltaPHI tiene **negativos**: normal, es una *diferencia* de porosidades.
- ⚠ Miren el count de PE: **3 232 de 4 149**. Faltan casi mil valores. Siguiente slide.

Datos Faltantes: una Decisión, No un Accidente

A 3 pozos no les corrieron la herramienta de PE. ¿Qué hacemos?

Python

```
print(df['PE'].isna().sum())
print(df[df['PE'].isna()]['Well
↳ Name'].unique())

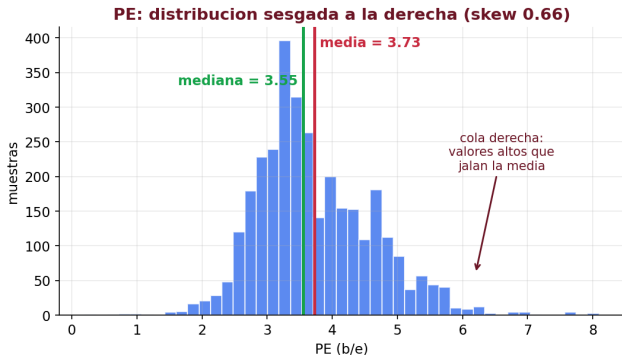
df = df.dropna().reset_index(drop=True)
print(df.shape)
```

>_ Output

```
917
['ALEXANDER D' 'KIMZEY A' 'Recruit F9']
(3232, 11)
```

- ? Opciones: **quitar las filas**, quitar la columna PE, o **inventar** el valor (imputar).
- ✓ Hoy: quitar filas. Es la opción **honesta y simple**, y nos quedan 3 232 muestras de 8 pozos.
- ⚠ Quitar PE sería tirar una de las mejores huellas de mineralogía — lo comprobaremos con las importancias.
- ➔ ¿Y la tercera opción — **imputar**? Rellenar el hueco con un valor razonable. Merece sus propias slides: van ya.

Imputar: Rellenar, Pero con Cabeza



→ Si van a inventar un valor, que sea uno **típico**. ¿Cuál? Depende de la **forma de la distribución**.

⚠ PE tiene **cola derecha**: los valores altos **jalan la media** hacia arriba (3.73). La **mediana** (3.55) no se deja arrastrar.

✓ Regla práctica: distribución **simétrica** → media o mediana da igual; **sesgada o con extremos** → **mediana**.

👁 Caso extremo en este dataset: GR (skew 2.0 por las lutitas calientes). Ahí la media sí mentiría.

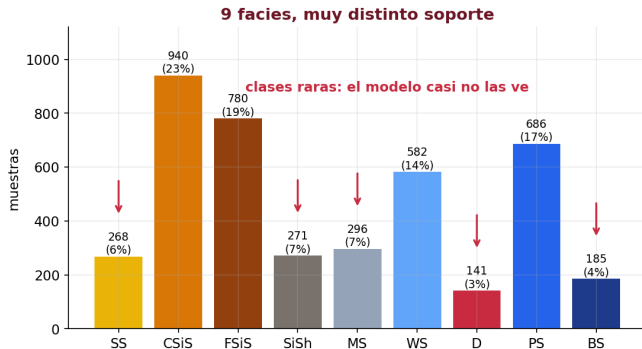
El Menú de la Imputación: Qué Usar en Cada Caso

Método	Qué hace	Cuándo elegirlo	Riesgo
Quitar filas	descarta muestras incompletas	faltan pocas filas y sobran datos	perder pozos o clases enteras
Quitar columna	descarta la variable	la variable aporta poco	botar señal valiosa (¡PE!)
Media	rellena con el promedio	distribución simétrica , faltante aleatorio	la arrastran los extremos
Mediana Por grupo	rellena con el valor central mediana <i>dentro</i> de cada formación o pozo	distribución sesgada (como PE) el grupo explica la variable	aplana la variabilidad grupos chicos → ruido
Con modelo	predice el faltante desde las otras variables (KNN, regresión)	falta mucho y la variable es clave	complejidad + fugas

✓ Lo probamos con nuestro RF: quitar filas **0.764**, mediana **0.762**, media **0.767**, mediana por formación **0.755**. Aquí **empatan** — el faltante era ordenado (3 pozos) y PE tiene sesgo moderado.

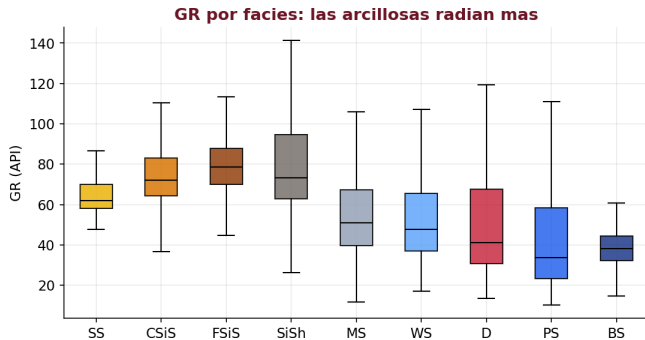
⚠ Detalle fino que dejamos sembrado: la media/mediana se calcula **solo con el train** — calcularla con todo es **fuga de información**. El porqué, en la Clase 5.

El Balance: 9 Clases, Soportes Muy Distintos



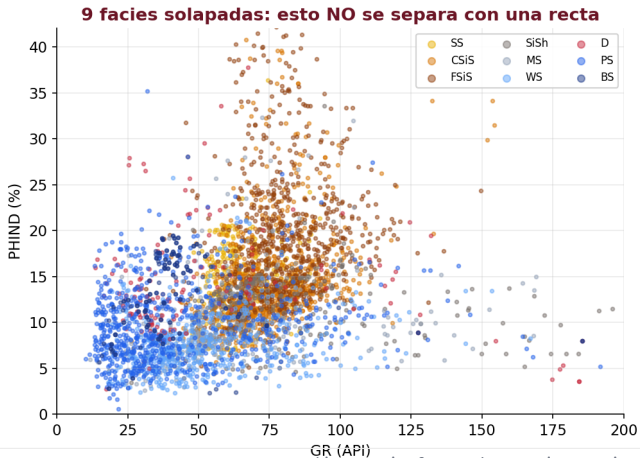
- La limolita gruesa (CSiS) tiene **940** muestras; la dolomita (D), apenas **141**.
- ⚠ En la Clase 2 el desbalance era 2 a 1. Aquí es **7 a 1** entre la clase más común y la más rara.
- 🎯 Pregunta para dentro de un rato: ¿el modelo aprenderá la dolomita... o la **ignorar**á?

Explorar por Clase: el GR Cuenta una Historia



- ✓ La física responde: las facies con más arcilla (FSiS, SiSh) **radian más**; los carbonatos limpios (PS, BS), menos.
- ➔ Pero miren los **solapes**: con GR solo, MS, WS y D son casi indistinguibles.
- 🎯 Una variable no alcanza: necesitamos que el modelo **combine** las cinco curvas + el contexto.

El Crossplot: por Esto Traemos al Bosque



→ 9 nubes de puntos **encimadas**. No hay recta — ni nueve rectas — que las separen limpio.

✓ Es el mismo mensaje de las **lunas** de la Clase 3, pero con datos reales y 9 colores.

🌲 Por eso el modelo de hoy vuelve a ser el **Random Forest**: cortes no lineales, sin exigir escalado.

Codificar: el Idioma de los Modelos

Los modelos solo comen números

Los Modelos Solo Comen Números

Todo lo que hay dentro de un modelo son sumas y comparaciones. “Dolomita” no se puede sumar.

USTEDES YA LO HACEN

Cuando llenan un formato de campo: ¿pozo fluyendo? **1**; ¿cerrado? **0**. ¿Tipo de levantamiento? marcan **una casilla** entre gas lift / BES / varilla.

Convertir categorías en casillas y unos-y-ceros es codificar. Solo falta ponerle nombre técnico.

- En nuestra tabla hay **3 columnas de texto o categoría**: el target Facies, el contexto NM_M y la Formation.
- Cada una necesita un **tratamiento distinto** — y equivocarse de tratamiento **daña el modelo** en silencio.
- ✓ En 4 slides tendrán el **mapa completo**: qué técnica, para qué columna, y por qué.

El Mapa Completo de Codificación

Tres técnicas. La pregunta clave es siempre: ¿cuántas categorías, y es target o feature?

	Binarizar	LabelEncoder	One-hot
Qué hace	2 categorías $\rightarrow$ 0 / 1	N nombres $\rightarrow$ 0...N-1	1 columna $\rightarrow$ N columnas de 0/1
Se usa en	features de 2 valores	el target multiclase	features de 3+ valores
Hoy	NM_M $\rightarrow$ Marino	Facies (9 nombres)	Formation (14 valores)
Ya la vimos	Clase 2 (lutita 0/1)	hoy	hoy

⚠ ¿Por qué no usar LabelEncoder en una **feature** de 14 valores? Porque le inventa un **orden falso** a las categorías. Eso merece su propia slide.

⚠ ¿Y por qué no one-hot en el **target**? También lo veremos: error clásico.

La Trampa del Orden Falso

Si numeramos arenisca = 1, caliza = 2, lutita = 3... el modelo se lo cree.



- Los números traen **orden y distancia** de regalo: 2 está entre 1 y 3, y $3 - 1 = 2$. Las categorías geológicas **no tienen** nada de eso.
- ⚠ Nuestro dataset ya viene con la trampa puesta: Facies llega como números 1–9. Por eso lo primero será **devolverles su nombre**.

La Prueba: ¿y si Tratamos la Facies como Número?

Hagamos el experimento del error: predecir el *número* de facies con regresión.

Python

```
from sklearn.ensemble import  
    RandomForestRegressor  
  
reg =  
    RandomForestRegressor(random_state=42)  
reg.fit(X_tr, y_tr)  
print(reg.predict(X_te)[:6].round(2))
```

>_ Output

```
[5.12 3.97 4.37 3.02 6.63 2.82]
```

- ✗ ¿Facies **4.37**? **No existe**. El modelo promedia categorías como si fueran barriles.
- ✗ Si redondeamos a la facies más cercana: accuracy **0.40**, contra **0.76** clasificando como se debe.
- ✓ Moraleja: número $\neq$ cantidad. Cuando la variable es una **categoría**, el problema es de **clasificación**. Siempre.

LabelEncoder: el Diccionario del Target

Para el target sí usamos números 0...8 — pero como *código*, no como cantidad.

Python

```
from sklearn.preprocessing import
    LabelEncoder

le = LabelEncoder()
y = le.fit_transform(df['FaciesNombre'])
print(le.classes_)
print(y[:3], '->',
      le.inverse_transform(y[:3]))
```

>_ Output

```
['BS' 'CSiS' 'D' 'FSiS' 'MS' 'PS' 'SS'
 'SiSh' 'WS']
[3 3 3] -> ['FSiS' 'FSiS' 'FSiS']
```

- Es un **diccionario de ida y vuelta**: `fit_transform` pone códigos, `inverse_transform` devuelve nombres.
- ✓ En el target no hay trampa ordinal: el clasificador trata cada código como **casilla separada**, nunca hace cuentas con ellos.
- Regla: LabelEncoder → **solo el target**. Para features categóricas, lo que sigue.

One-Hot: una Casilla por Categoría

Para features categóricas: cada valor se vuelve su propia columna de 0/1. Sin orden, sin trampa.

Python

```
oh = pd.get_dummies(df['Formation'],  
                    prefix='Fm')  
print(oh.shape[1], 'columnas nuevas')  
print(list(oh.columns[:3]))  
  
df['Marino'] = (df['NM_M'] == 2).astype(int)
```

>_ Output

```
14 columnas nuevas  
['Fm_A1 LM', 'Fm_A1 SH', 'Fm_B1 LM']
```

- ❏ Formation (14 valores) → **14 columnas**. Cada fila tiene un solo 1: su casilla.
- ✓ NM_M tiene solo 2 valores → basta **binarizar**: Marino = 0/1, igual que la lutita de la Clase 2.
- ⚠ El precio del one-hot: **más columnas**. Con 14 valores va bien; con 10 000 (un ID de pozo) sería una mala idea.

El Error Clásico: One-Hot en el Target

“Si one-hot es tan bueno... ¿por qué no se lo aplico a Facies?”

✗ LO QUE PASA SI LO HACEN

El target deja de ser una columna y se vuelve 9. scikit-learn cree que son **9 problemas independientes** de sí/no... y puede responder “sí” a dos facies a la vez, o a ninguna.

Una muestra tiene **exactamente una** facies: el target debe ser **una sola columna** de códigos.

- ✓ Regla de bolsillo, para pegar en el monitor:
- **Target** categórico → `LabelEncoder` (una columna de códigos).
- **Feature** con 2 valores → binarizar (0/1).
- **Feature** con 3+ valores → one-hot (`get_dummies`).
- ⊘ Números ordinales inventados en features → **nunca**, salvo que el orden sea real (p. ej. calidad mala/media/buena).

¿Y Todo Este Esfuerzo Paga?

Midámoslo: mismo modelo, con y sin las columnas codificadas.

Modelo	Solo 6 registros	+ Marino + one-hot	Ganancia
Logística multiclase	0.511	0.594	+8.3 pts
Random Forest	0.697	0.764	+6.7 pts

- ✓ El **contexto geológico** (marino/no marino, formación) que ustedes darían por obvio, el modelo **no lo ve** hasta que se lo codifican.
- Codificar bien no es burocracia: son **6–8 puntos de accuracy gratis** — más que muchos cambios de modelo.

El Bosque, Ahora con 9 Clases

El mismo código, un problema más grande

Random Forest Multiclase: el Mismo Código de la Clase 3

Prometido: el flujo no cambia. `scikit-learn` cuenta las clases de `y` y listo.

Python

```
from sklearn.ensemble import  
    RandomForestClassifier  
  
rf = RandomForestClassifier(  
    n_estimators=200, random_state=42)  
rf.fit(X_tr, y_tr)  
print(round(rf.score(X_te, y_te), 3))
```

>_ Output

0.764

- ✓ **76.4 %** de aciertos exactos entre **9 opciones**. El azar puro daría 11 %; el modelo tonto, 23 %.
- Cada árbol vota por **una de las 9 facies**; gana la mayoría. La misma democracia de la Clase 3.
- 👁 *¿76 % es bueno?* Con accuracy sola **no se puede saber**. Necesitamos abrir la caja — sección siguiente.

El Marcador: Tres Modelos, Mismo Test

Como siempre: cualquier modelo se compara contra el tonto y contra el lineal.

Modelo	Accuracy	F1 macro	F1 weighted
Modelo tonto (siempre CSiS)	0.229	0.041	0.085
Logística multiclase (escalada)	0.594	0.573	0.592
Random Forest (200 árboles)	0.764	0.754	0.763

- ✓ La **no linealidad** vuelve a pagar: +17 puntos sobre la recta, igual que nos pasó con las lunas y la litología en la Clase 3.
- ➔ Aparecieron dos columnas nuevas en el marcador: **F1 macro** y **F1 weighted**. Explicarlas — y entender a quién protege cada una — es el corazón de lo que sigue.

Leer 9 Clases a la Vez

La matriz de confusión ahora es 9×9

La Matriz de Confusión, Versión 9×9

Matriz de confusion 9x9 del Random Forest (test)

	SS	CSIS	FSIS	SiSh	MS	WS	D	PS	BS
SS	56	21	1	0	0	0	0	0	0
CSIS	6	184	31	0	0	1	0	0	0
FSIS	0	29	152	1	0	1	0	2	0
SiSh	0	0	0	37	2	13	0	3	0
MS	0	1	3	2	36	10	2	11	0
WS	0	0	0	4	8	107	1	19	0
D	0	0	1	0	0	3	18	5	2
PS	0	2	3	3	1	27	3	109	1
BS	0	0	0	1	0	0	0	5	42
	SS	CSIS	FSIS	SiSh	MS	WS	D	PS	BS

Facies predicha

→ Se lee igual que la 2×2 de la Clase 2:
fila = lo que era, columna = lo que dijo el modelo. La diagonal son los aciertos.

✓ Ordenamos las facies **geológicamente** (no marinas primero, marinas después) y aparece algo: los errores se **pegan a la diagonal**.

👁 Cada casilla fuera de la diagonal es una historia: ¿*por qué* confundió 31 limolitas gruesas con finas?

🎯 Con 9 clases, esta matriz **es** el reporte del modelo. La accuracy es solo su **resumen de una línea.**

Leerla con Ojos de Geólogo

Pregunta obligada: ¿los errores del modelo tienen sentido físico?

Las confusiones más frecuentes del bosque:

Real → predicha	Casos	¿Vecinas?
CSiS → FSiS	31	✓
FSiS → CSiS	29	✓
PS → WS	27	✓
SS → CSiS	21	✓
WS → PS	19	✓

- ✓ El modelo confunde **limolita gruesa con fina y packstone con wackestone**: exactamente lo que confundiría un geólogo con el registro en la mano.
- ➔ De los 229 errores del bosque, **el 67 % cae en una facies geológicamente vecina**.
- ⊙ Un modelo que se equivoca “con sentido” inspira más confianza que uno que acierta más pero delira cuando falla.

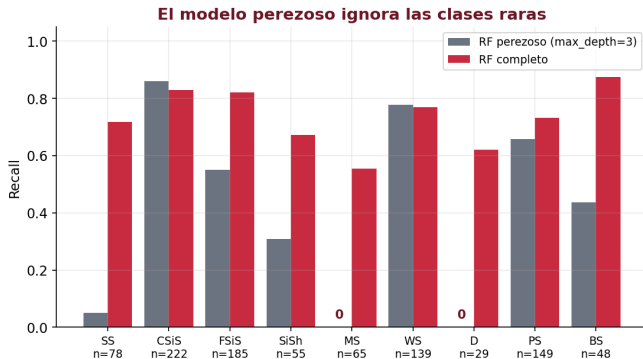
Precision y Recall, Ahora por Clase

Las definiciones de la Clase 2 no cambian — solo que ahora hay un par por facies.

Facies	Soporte	Prec.	Recall	F1
SS	78	0.90	0.72	0.80
CSiS	222	0.78	0.83	0.80
FSiS	185	0.80	0.82	0.81
SiSh	55	0.77	0.67	0.72
MS	65	0.77	0.55	0.64
WS	139	0.66	0.77	0.71
D	29	0.75	0.62	0.68
PS	149	0.71	0.73	0.72
BS	48	0.93	0.88	0.90

- **Recall de MS** = de todos los mudstone reales, ¿qué fracción encontró? Solo el **55 %**.
- **Precision de MS** = cuando dice mudstone, ¿qué tan confiable es? El 77 %.
- ⚠ Las clases en rojo son las de **soporte chico**: el modelo las aprende peor. El desbalance de la exploración **ya está cobrando**.
- ✓ En sklearn, todo esto sale de una línea: `classification_report()`.

El Modelo Perezoso: la Ceguera a las Clases Raras



- Entrenamos a propósito un bosque **recortado** (`max_depth=3`): accuracy **0.56** — suena aceptable.
- ✗ Pero miren: mudstone y dolomita, **recall = 0.00**. Para el modelo, **no existen**.
- ⚠ Es la estrategia perezosa: apostar siempre a las clases grandes. La accuracy **lo premia**; las clases raras **lo pagan**.

Dos Maneras de Promediar 9 Notas

Tenemos 9 valores de F1 — uno por facies. ¿Cómo los resumimos en un solo número?

⚖️ WEIGHTED (ponderado)

Cada clase pesa según su **tamaño**: la nota de CSiS (222 muestras) manda; la de D (29) casi no cuenta.

Protege a las clases grandes. Se parece a la accuracy.

⚖️ MACRO (parejo)

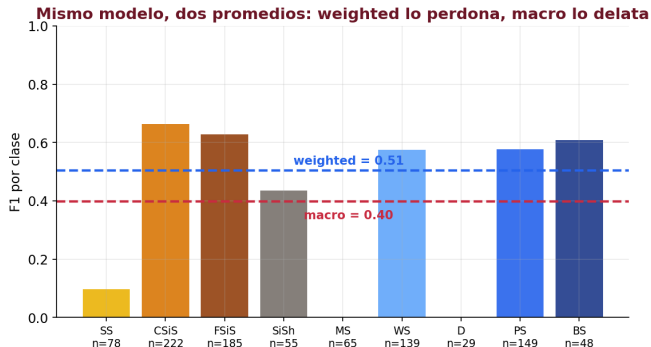
Promedio simple: **cada clase vale igual**, tenga 29 o 222 muestras. La dolomita opina con la misma voz que la limolita.

Protege a las clases raras.

📊 Cuenta rápida con 2 clases: una grande (900 muestras, $F1 = 0.90$) y una rara (100 muestras, $F1 = 0.10$).
Weighted = $0.9 \times 0.90 + 0.1 \times 0.10 = \mathbf{0.82}$. Macro = $(0.90 + 0.10)/2 = \mathbf{0.50}$. **El mismo modelo.**

➔ ¿Cuál reportar? Depende de **qué les duele perder**. Si la clase rara es la que importa (la dolomita productora...), macro.

Mismo Modelo, Dos Veredictos



- ➔ El modelo perezoso otra vez: F1 **weighted 0.51** (“regular”), F1 **macro 0.40** (“mal”).
- 👁 La brecha entre los dos números es un **detector de clases abandonadas**: si $\text{macro} \ll \text{weighted}$, alguien quedó en cero.
- ✓ En nuestro bosque completo van parejas (0.754 vs 0.763): ninguna facies quedó abandonada del todo.

Su Turno: la Logística Bajo la Lupa

🕒 15 min

El bosque ya rindió cuentas clase por clase. Ahora háganle la misma auditoría a la logística.

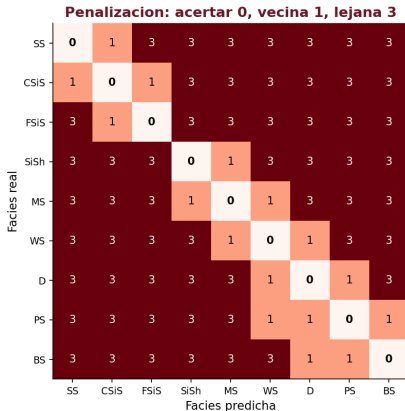
1. Entrenen la **logística multiclase** (con `StandardScaler`, como en la Clase 2) sobre las mismas features codificadas.
2. Saquen su `classification_report`. ¿Qué facies tienen **recall** por debajo de 0.40?
3. Comparen su **F1 macro** contra su **F1 weighted**. ¿La brecha delata clases abandonadas?
4. Pregunta de negocio: si el cliente pregunta “¿qué tan bueno es el modelo?”, ¿qué número le darían y **por qué ese**?

Pista: en el notebook está la celda lista con # Escribe tu solución aquí.

No Todos los Errores Cuestan Igual

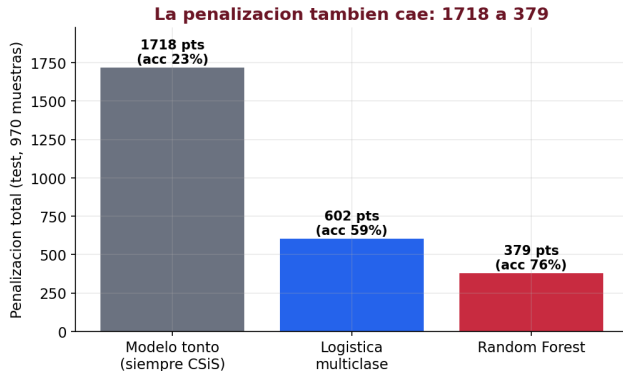
El costo de negocio, en 9 dimensiones

La Matriz de Penalización



- ➔ En la Clase 2 el costo tenía 2 números (FP y FN). Con 9 clases necesita una **tabla completa**: cuánto duele confundir *cada* facies con *cada* otra.
- ✓ Nuestra regla, simple y geológica: acertar **0**, facies **vecina 1** (limolita gruesa por fina: molesto), facies **lejana 3** (caliza por arenisca: grave).
- 🏆 No lo inventamos nosotros: la competencia SEG calificaba con una matriz así — premiaba fallar cerca.
- 🎯 La matriz de penalización es la **traducción del negocio** al modelo: aquí es donde el ingeniero opina.

El Récord de Hoy: Penalización Total



- ➔ Se calcula multiplicando **casilla por casilla**: matriz de confusión $\times$ matriz de penalización, y sumando.
- ✓ El bosque no solo acierta más: **cuando falla, falla cerca**. Por eso su penalización cae a menos de la cuarta parte del tonto.
- 🏆 Récord del módulo, capítulo 3: en litología binaria fue 5 953 $\rightarrow$ 1 150 (pesos); hoy es **1718 $\rightarrow$ 379 puntos**.

Cómo Reportarlo al que Decide

El gerente no quiere oír “F1 macro 0.754”. Quiere saber si puede confiar en el mapa de facies.

✗ ASÍ NO

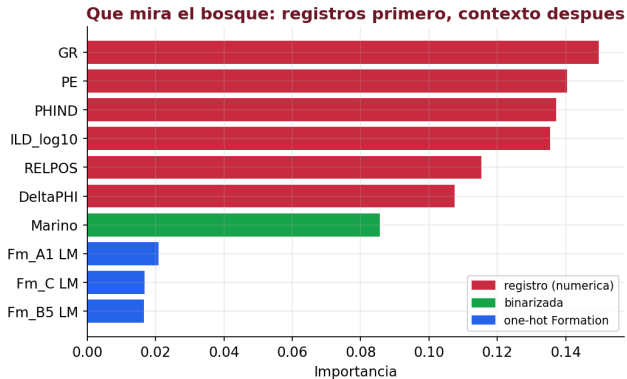
“El clasificador alcanzó un accuracy de 0.764 con F1 macro de 0.754 y una penalización agregada de 379 unidades sobre el conjunto de prueba.”

✓ ASÍ SÍ

“De cada 100 pies, el modelo acierta la facies exacta en **76** y en **92** queda a lo sumo en la vecina. Falla sobre todo entre limolitas — el mismo punto ciego de un intérprete con solo registros.”

- El número **92 de cada 100** sale de contar como aceptable la facies vecina: accuracy 0.764 exacta, **0.923** relajada.
- Digan también **dónde no confiar**: mudstone y dolomita (poco soporte, recall 0.55–0.62) ameritan revisión humana.
- ✓ Esto es lo que separa al ingeniero del que solo corre `.fit()`: traducir métricas a **decisiones con límites claros**.

¿Qué Miró el Bosque para Lograrlo?



✓ Los **5 registros** dominan — y PE, el que casi botamos por sus faltantes, es el **segundo** más usado.

➔ Las columnas codificadas (Marino, one-hot de Formation) aportan menos por separado... pero ya vimos que en conjunto valen **+6.7 puntos**.

⚠ Recordatorio de la Clase 3: importancia $\neq$ causalidad. Es **uso**, no física.

Las Perillas del Modelo

Parámetros vs hiperparámetros

Dos Tipos de Números Viven en un Modelo

Ya usamos `max_depth` y `n_estimators` varias veces. Pongámosles apellido.

⚙️ PARÁMETROS

Los aprende **el modelo solo**, desde los datos, durante `.fit()`.

Ejemplos: los **cortes** de cada árbol ("¿GR > 78?"), los coeficientes de la logística.

Ustedes **nunca los escriben**.

≡ HIPERPARÁMETROS

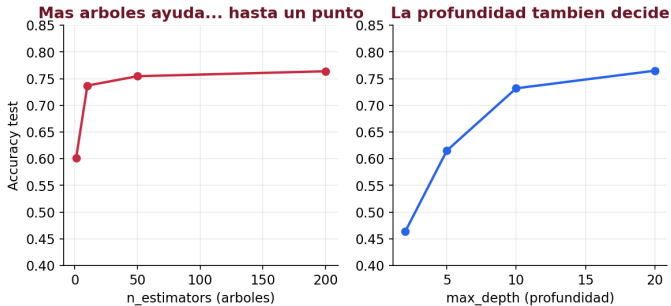
Los eligen **ustedes**, *antes* de entrenar.

Ejemplos: `max_depth`, `n_estimators`, el `test_size` del split.

Son las **perillas** del tablero: el modelo no puede girarlas por sí mismo.

→ Analogía de taladro: el **peso sobre la broca** lo ajusta el perforador (perilla); el **desgaste** que va tomando la broca lo pone la formación (aprendido). Uno se decide, el otro resulta.

Girando las Perillas a Mano



- Con 10 árboles ya casi todo está ganado; de 50 a 200 se rasca **un punto**.
- max_depth recortado **cuesta caro** (ya vimos al perezoso).
- ❓ Pregunta incómoda: aquí elegimos perillas **mirando el test...** ¿no es eso **hacer trampa**?

Sí lo es. La herramienta que lo hace bien viene en la siguiente slide.

GridSearchCV: la Búsqueda Automática de Perillas

Le damos la parrilla de valores; él prueba todas las combinaciones — sin tocar el test.

Python

```
from sklearn.model_selection import
    GridSearchCV

parrilla = {'max_depth': [5, 10, 20, None],
            'n_estimators': [50, 200]}
grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    parrilla, cv=5)
grid.fit(X_tr, y_tr)
print('mejor:', grid.best_params_)
```

>_ Output

```
mejor: {'max_depth': None,
        'n_estimators': 50}
```

- Grid = parrilla: $4 \times 2 = 8$ combinaciones de perillas.
- cv=5: cada combinación se valida **5 veces dentro del train** (por eso el test queda intacto). Son $8 \times 5 = 40$ entrenamientos... hechos por él.
- best_params_: la combinación ganadora.
best_score_: su accuracy **de validación** (no de test).

Qué Nos Dijo la Búsqueda

La parrilla completa, con su accuracy de validación (promedio de los 5 pliegues).

max_depth	n_estimators	Validación
5	50	0.614
5	200	0.622
10	50	0.707
10	200	0.711
20	50	0.746
20	200	0.755
None	50	0.755
None	200	0.748

- La **profundidad** manda: de 5 a 20 niveles se ganan 14 puntos. Los árboles (50 vs 200) mueven **décimas**.
- 👁 Los dos primeros lugares empatan (0.755): a partir de cierta profundidad, **todo da más o menos igual**.
- ✓ El ganador en test: **0.755** — prácticamente nuestro 0.764 de siempre. Moraleja honesta: **no siempre hay tesoro escondido**; la búsqueda sirve para *confirmarlo sin hacer trampa*.
- ❓ ¿Por qué 5 pliegues? ¿Por qué funciona validar dentro del train? Eso **es** la Clase 5.

Su Turno: su Propia Matriz de Penalización

🕒 15 min

Nuestra matriz (0/1/3) es geológica. Háganla de negocio.

1. Supongan que las facies de **buena roca almacén** son SS, PS y BS. Construyan una matriz donde confundir roca almacén con roca sello (o al revés) cueste **5**, y cualquier otro error cueste **1**.
2. Calculen la **penalización total** del Random Forest y de la logística con *su* matriz. ¿Cambia el ganador?
3. Encuentren en la matriz de confusión **la casilla que más plata les cuesta** con su regla. ¿Entre qué facies está?
4. Pregunta de negocio: escriban **una frase para el gerente** de activos recomendando (o no) usar el modelo para mapear roca almacén.
5. **Bonus** — con GridSearchCV, agreguen `min_samples_leaf` $\in \{1, 5, 20\}$ a la parrilla. ¿Cambia el ganador?

Cierre

Lo que se llevan hoy

Lo Que Aprendimos Hoy

Conceptos

- ✓ Multiclase: elegir una entre N (9 facies)
- ✓ Faltantes e imputación: el menú y cómo elegir según la **distribución**
- ✓ Codificar: binarizar / LabelEncoder / one-hot — y cuándo usar cada una
- ✓ La trampa del orden falso (facies 4.37 no existe)
- ✓ Matriz 9×9 léida **geológicamente**
- ✓ Macro vs weighted: a quién protege cada promedio
- ✓ Matriz de penalización: el costo en 9 dimensiones
- ✓ Parámetros vs hiperparámetros (las perillas)

Herramientas

- ✗ LabelEncoder (+ inverse_transform)
- ✗ pd.get_dummies (one-hot)
- ✗ classification_report, confusion_matrix
- ✗ f1_score(average='macro' / 'weighted')
- ✗ fillna (mediana), GridSearchCV

El número del día:

76 de 100 pies con la facies **exacta**; 92 de 100 a lo sumo en la **vecina**. Penalización: tonto **1718** → bosque

379.

Lo Que Sigue: el Cierre del Módulo

Clase 5: un modelo nuevo con una idea hermosa, y la respuesta a la pregunta incómoda.

- ➔ **SVM y el truco del kernel** — separar lo inseparable subiendo de dimensión (las lunas de la Clase 3 vuelven por última vez).
- ➔ **El porqué de la validación cruzada** — hoy usamos GridSearchCV como receta; en la Clase 5 abrimos el $cv=5$ y entendemos por qué protege del sobreajuste de perillas.
- ➔ **La deuda pendiente** — validar con **pozos completos por fuera**: la anotamos en la Clase 2 y en la 3; en la 5 por fin se paga.

*Llegan a la Clase 5 sabiendo entrenar, codificar, evaluar y costear. Falta aprender a **afinar y validar como profesionales.***

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

SLB Ecuador · UDLA · 2026